How to create a new spell safely

Below is a **step-by-step Unreal Editor** guide:

1. Spell identity via GameplayTags
2. AbilityInfo data
3. Grant assigns InputTag + CooldownTag
4. ASC activates by InputTag
5. Damage via GameplayEffects
6. Health via AttributeSet.

Example spell: **Fireball** (active projectile, single target with optional burn).

Fireball spell data example

Mechanics

1. Player receives Fireball (server):
   - Fireball is put into next free active slot (e.g., slot 1)
   - It is granted with:
     - InputTag = InputTag.1
     - CooldownTag = Cooldown.Slot.1
     - CooldownEffect = GE_Cooldown_Slot1

2. Player presses key
   1
   :
   - Enhanced Input triggers action bound to InputTag.1
   - PlayerController forwards InputTag.1 to ASC
   - ASC finds the spec tagged InputTag.1 and activates Fireball

3. Fireball spawns projectile (server), projectile overlaps enemy:
   - Applies GE_Damage_Fireball (and any additional effects) via the helper library
   - ExecCalc computes final damage, AttributeSet subtracts from Shield then Health

Naming and tags

- Spell tag: Spell.Fireball
- Damage type: Damage.Fire
- Ability BP: GA_Fireball
- Damage GE: GE_Damage_Fireball

- Slot cooldown: GE_Cooldown_Slot1 grants Cooldown.Slot.1 (or your actual cooldown tag)

AbilityInfo row fields

Field	Value
AbilityTag	Spell.Fireball
AbilityType	Spell.Type.Active
Ability	GA_Fireball
UI Name/Icon	“Fireball” / fireball icon

Asset checklist

Asset / thing	Example name	Purpose
GameplayTag (Spell identity)	Spell.Fireball	The spell’s canonical ID used in AbilityInfo and lookups
GameplayTag (Damage type)	Damage.Fire (if you have it)	Drives resistances/debuff mappings in damage calculation
GameplayAbility class (BP or C++)	GA_Fireball	Activation logic (spawn projectile / apply effects)
Damage GameplayEffect	GE_Damage_Fireball	Executes damage calculation / applies IncomingDamage
Cooldown GameplayEffect	GE_Cooldown_Fireball	Applies the cooldown tag so UI can track cooldown
AbilityInfo entry (DataAsset/ DataTable row)	Row for Spell.Fireball	Links Spell tag → ability class → type (Active/Passive/Special)
(Optional) Projectile actor BP	BP_Projectile_Fireball	Collision + movement + on-hit VFX (if you use actors for spells)
(Optional) Niagara/VFX/SFX	NS_FireballTrail, etc.	Presentation

---

Step-by-step

1) Create/verify GameplayTags (spell identity + damage type)

Editor path: Edit → Project Settings → Gameplay Tags

1. In
   Gameplay Tag List
   , add:
   - Spell.Fireball

2. Ensure you have an appropriate damage type tag (only if you use damage-type scaling):
   - Damage.Fire (or whatever your project uses for fire)

Naming convention:

- Spell identity tags: Spell.<Name>
- Damage type tags: Damage.<Type>

Don't break: the spell identity tag is the "primary key" for AbilityInfo and most lookups.

---

2) Create the GameplayAbility (Blueprint child of your base ability)

Editor path: Content Browser Add (+) Blueprint Class

1. Pick **All Classes**
2. Search for your base ability class (UBaseGameplayAbility or it's inheritors)
3. Create blueprint:
   - GA_Fireball

Now configure it:

2.1 Set the spell's identity asset tag(s)

In the GA blueprint: open **Class Defaults** and find **AssetTags** tag settings.

- Add
  exactly one
  identifying tag:
  - Spell.Fireball

Why: parts of code assume the ability's identity is GetAssetTags().First(). **What not to do:** don't add 3–5 different spell tags and hope it "figures it out".

2.2 Choose how the spell gets an InputTag

We have **two granting pathways**:

- **Granted as a "spell" via PlayerState**: InputTag comes from the slot assignment when the server grants it. In this case, **do not** set a fixed StartupInputTag in the ability.
- **Granted as a "startup ability" from a character's StartupAbilities**: you must set the ability's **StartupInputTag** (e.g., InputTag.1). Use this for always-available baseline abilities.

For Fireball (a normal active spell you pick/receive), assume **PlayerState grant** assigns InputTag by slot.

---

3) Create the Damage GameplayEffect (GE)

Editor path: Content Browser Add (+) Blueprint Class (or Gameplay Effect asset)

Create:

- GE_Damage_Fireball

Configure it so it participates in your unified damage pipeline:

3.1 Instant vs duration

- **Instant** : This effect applies instantly.
- **Infinite** : This effect lasts forever.
- **HasDuration** : The duration of this effect will be specified by a magnitude.

3.2 Execution calculation

- Add **Damage ExecCalc** (execution calculation) to the GE.
- Ensure the GE writes to the appropriate meta-attribute like GameAttributeSet.Mana

3.3 SetByCaller magnitudes (damage amount + type)

The damage exec expects SetByCaller magnitudes keyed by tags like damage type and debuff parameters. So Fireball should provide at least:

- SetByCaller magnitude for Damage.Fire (or the chosen damage type tag)
- optionally debuff SetByCaller tags (chance/duration/frequency/damage) if you want burn

Note: The GE must be authored to *use* the execution and read SetByCaller values in the exec calc.

---

4) Create the Cooldown GameplayEffect (GE)

Create:

- GE_Cooldown_Fireball

Critical wiring requirement: the cooldown GE must grant (or carry as asset tag) the **CooldownTag** that your UI and cooldown-watcher listens for.

The system is slot-based, so in practice:

- Fireball doesn't "own" a cooldown tag; the **slot** does.
- When the spell is granted, the server chooses CooldownTag from the slot (e.g., "Slot1 cooldown").

So we have two common patterns:

Pattern A (recommended): One cooldown GE per slot

- GE_Cooldown_Slot1 grants Cooldown.Slot.1 (or your project's actual slot tag)
- GE_Cooldown_Slot2 grants Cooldown.Slot.2, etc. Then when a spell is put into slot 1, it uses slot 1 cooldown GE.

Pattern B: One cooldown GE per spell (only if cooldown tag is ability-based)

- GE_Cooldown_Fireball grants Cooldown.Spell.Fireball This works only if your PlayerState / UI is wired to that tag. The current design is more slot-oriented.

What NOT to break: if the cooldown GE doesn't grant the tag the UI is watching, cooldown UI won't update.

---

5) Add the spell to AbilityInfo (data definition)

This is the piece that makes Spell.Fireball “known to the game”.

Editor path: locate DA_AbilityInfo in Content Browser.

Add an entry/row:

- **AbilityTag:** Spell.Fireball
- **AbilityType:** Spell.Type.Active
- **Ability:** GA_Fireball (class reference)
- Any additional fields your UI needs (name, icon, description, rarity, etc.)

Why: granting by tag (and categorizing passive vs active vs special) depends on this data.

---

6) Implement activation: spawn projectile or apply effects

Create:

- BP_Projectile_Fireball (child of your projectile base actor)

AActor
 ABaseAbilityActor
 AAuraWithMagicCircleActor
 AProjectileActor
  AAuraFireBall
 AGameEffectActor

In the projectile BP:

- assign collision profile / overlap
- set visuals (mesh/niagara)
- set movement speeds (or let ability set params)

In GA_Fireball ActivateAbility:

- spawn the projectile on the **server**
- pass in:
  - damage params (base damage, damage type, GE classes list, debuff params)
  - projectile behavior params (speed, lifetime, homing, etc.)

- set instigator/owner so hit validation and target filtering work

Note: For instant spells you may not need to spawn a projectile. Instead, in ActivateAbility:

- do a trace / target selection
- build and apply your GE container to targets

Check for silent failures

Failure	Cause
Ability has a stable single identity tag (Spell.*) in its asset tags	Ability lookup/type resolution can fail or use the wrong tag
Granted ability spec must have the correct InputTag dynamic tag	Input won't activate the ability (looks like "input broken")
Cooldown GE must grant the exact CooldownTag the UI watches	Cooldown UI never updates / cooldown events never fire
Damage should flow through IncomingDamage + AttributeSet handler	You get double damage, inconsistent shields, or no debuffs/knockback/death hooks
Passive/Special abilities must be safe to auto-activate on grant	Granting passives causes unexpected spawns/ loops/crashes

---

Safety checklist

Step	Done?	Notes
Create Spell. tag	☐	Project Settings → Gameplay Tags
Create/choose Damage. tag	☐	Optional but recommended
Create GA_ derived from base ability	☐	Add single spell identity asset tag
Create GE_Damage_	☐	Instant + ExecCalc + SetByCaller support
Create cooldown GE(s) that grant the slot cooldown tag	☐	Slot-based recommended
Add AbilityInfo entry for Spell.	☐	Links tag → ability class → type
Implement ActivateAbility (server spawn/ apply)	☐	Call Super unless intentionally skipping base behavior
Test: grant → input triggers → cooldown UI updates → damage applies	☐	Test in listen-server PIE if possible